

3주차 Python 라이브러리 실습 보고서

컴퓨터공학과 | 2022086026 | 심현석

NumPy와 Pandas 실습 6개, 3단 누적 막대그래프 Quiz, Matplotlib 산점도 실습을 정리하였다. 각 실습의 코드와 실행 결과를 비교하고, 결과가 나오는 이유를 간단히 설명하였다. 실행 환경은 macOS의 Python 3이며, 코드는 hw_1 폴더에서 실행하였다.

1 NumPy 및 Pandas 실습

실습 1 NumPy 배열 생성

리스트를 NumPy 배열로 바꾸고, 1차원 배열과 2차원 배열의 출력 형태를 확인하였다.

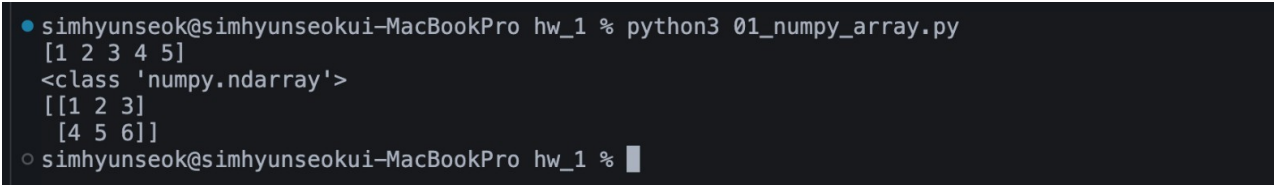
Python 코드

```
import numpy as np

ar1 = np.array([1, 2, 3, 4, 5])
print(ar1)
print(type(ar1))

ar2 = np.array([[1, 2, 3], [4, 5, 6]])
print(ar2)
```

수행 결과



```
simhyunseok@simhyunseokui-MacBookPro hw_1 % python3 01_numpy_array.py
[1 2 3 4 5]
<class 'numpy.ndarray'>
[[1 2 3]
 [4 5 6]]
simhyunseok@simhyunseokui-MacBookPro hw_1 %
```

그림 1 NumPy 배열 생성 실행 결과

결과 설명

np.array()에 한 개의 리스트를 전달하면 1차원 배열이 만들어지고, 리스트 안에 리스트를 넣으면 2차원 배열이 만들어졌다. type(ar1)의 결과는 numpy.ndarray였다. ar2에는 원소 6개가 2행 3열로 들어 있다.

실습 2 NumPy arange와 reshape

일정한 간격의 수열을 만들고, 같은 원소를 다른 행과 열의 구조로 배치해 보았다.

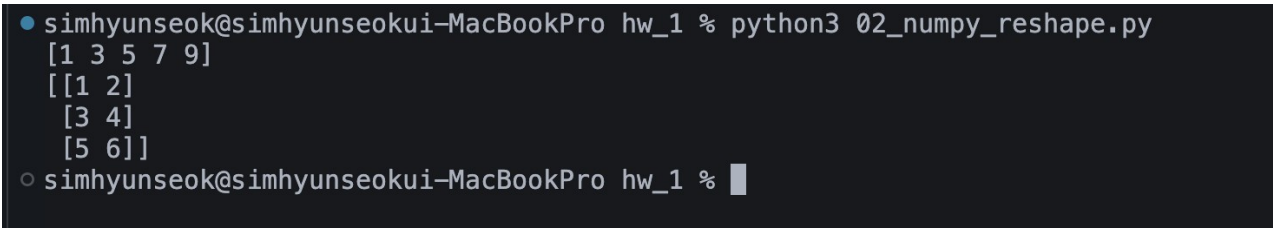
Python 코드

```
import numpy as np

ar1 = np.arange(1, 10, 2)
print(ar1)

ar2 = np.arange(1, 7, 1)
ar2 = ar2.reshape(3, 2)
print(ar2)
```

수행 결과



```
simhyunseok@simhyunseokui-MacBookPro hw_1 % python3 02_numpy_reshape.py
[1 3 5 7 9]
[[1 2]
 [3 4]
 [5 6]]
simhyunseok@simhyunseokui-MacBookPro hw_1 %
```

그림 2 NumPy arange와 reshape 실행 결과

결과 설명

np.arange(1, 10, 2)는 1부터 시작해 2씩 증가하며, 종료값 10은 포함하지 않는다. 따라서 1, 3, 5, 7, 9가 출력되었다. 두 번째 배열은 1부터 6까지의 원소를 reshape(3, 2)로 바꿔 3행 2열로 출력하였다. reshape를 적용해도 원소의 개수와 순서는 유지된다.

실습 3 NumPy 배열 슬라이싱

2차원 배열에서 원하는 행과 열만 선택하는 방법을 실습하였다.

Python 코드

```
import numpy as np

ar1 = np.arange(10, 70, 10)
ar1 = ar1.reshape(2, 3)
print(ar1[0:2, 0:2]) # 모든 행에서 앞 2열만 추출
print()
print(ar1[0, :]) # 첫 번째 행 추출
```

수행 결과

```
simhyunseok@simhyunseokui-MacBookPro hw_1 % python3 03_numpy_slicing.py
[[10 20]
 [40 50]]

[10 20 30]
simhyunseok@simhyunseokui-MacBookPro hw_1 %
```

그림 3 NumPy 배열 슬라이싱 실행 결과

결과 설명

`ar1[0:2, 0:2]`는 두 행에서 앞의 두 열만 선택하므로 `[[10, 20], [40, 50]]`이 된다. `ar1[0, :]`는 첫 번째 행의 모든 열을 선택하여 `[10, 20, 30]`을 반환한다. 슬라이싱 범위의 끝 인덱스는 포함되지 않는다는 점을 확인하였다.

실습 4 Pandas Series 생성

숫자와 문자열로 Series를 만들어 인덱스, 값, 자료형이 어떻게 표시되는지 확인하였다.

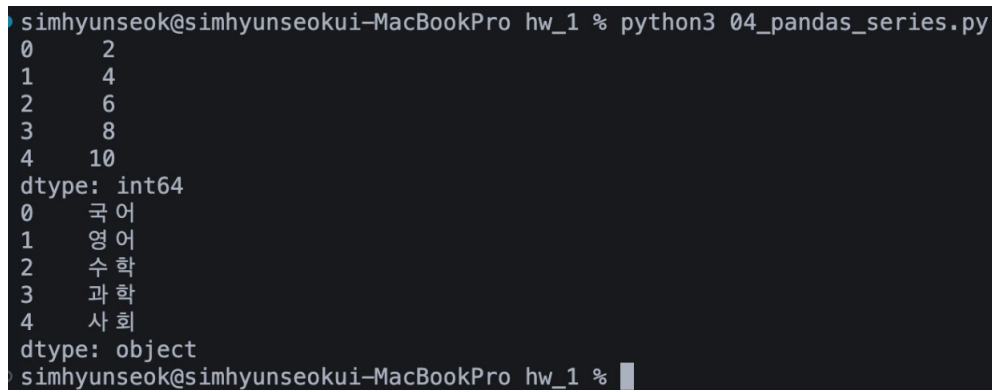
Python 코드

```
import pandas as pd

evenNum = pd.Series([2, 4, 6, 8, 10])
print(evenNum)

subjects = pd.Series(["국어", "영어", "수학", "과학", "사회"])
print(subjects)
```

수행 결과



```
simhyunseok@simhyunseokui-MacBookPro hw_1 % python3 04_pandas_series.py
0      2
1      4
2      6
3      8
4     10
dtype: int64
0   국 어
1   영 어
2   수 학
3   과 학
4   사 회
dtype: object
simhyunseok@simhyunseokui-MacBookPro hw_1 %
```

그림 4 Pandas Series 생성 실행 결과

결과 설명

인덱스를 따로 지정하지 않아 0부터 4까지의 정수 인덱스가 자동으로 붙었다. 숫자 Series와 과목명 Series 모두 같은 방식으로 값을 보관하지만, 저장한 값에 따라 자료형이 달라졌다. 실행 캡처에서는 숫자는 int64, 문자열은 object로 표시되었다. 문자열 자료형 표시는 Pandas 버전에 따라 str로 나타날 수도 있다.

실습 5 Pandas Series 인덱스 활용

과목명을 인덱스로 지정하여 점수를 조회하고, 인덱스와 값만 따로 확인하였다.

Python 코드

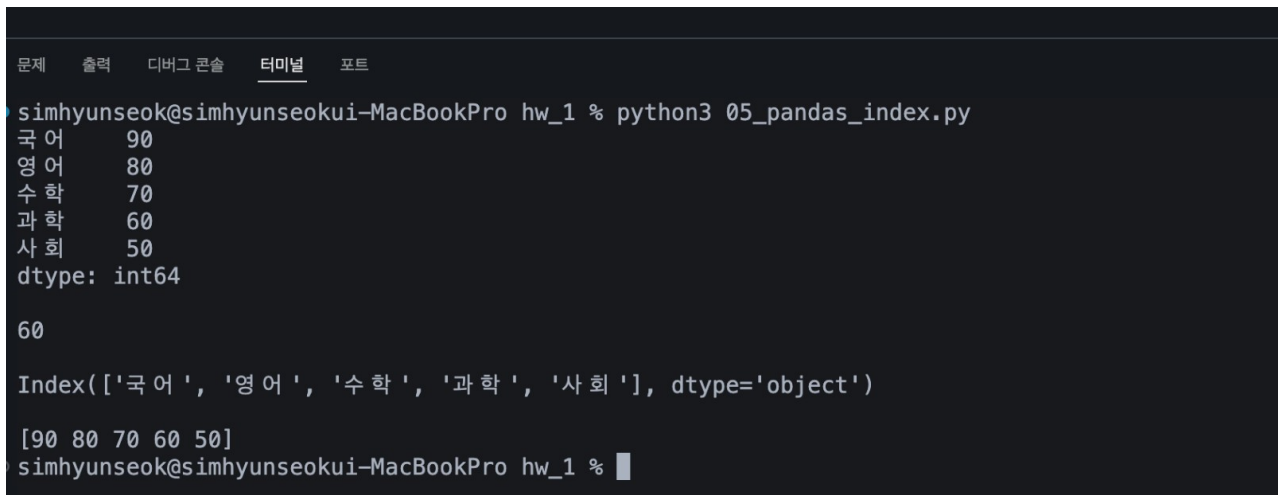
```
import pandas as pd

scores = pd.Series([90, 80, 70, 60, 50])

subjects = pd.Series(["국어", "영어", "수학", "과학", "사회"])

scoreSeries = pd.Series(data=scores.values, index=subjects.values)
print(scoreSeries)
print()
print(scoreSeries["과학"])
print()
print(scoreSeries.index)
print()
print(scoreSeries.values)
```

수행 결과



```
문제  출력  디버그 콘솔  터미널  포트
simhyunseok@simhyunseokui-MacBookPro hw_1 % python3 05_pandas_index.py
국어      90
영어      80
수학      70
과학      60
사회      50
dtype: int64

60

Index(['국어 ', '영어 ', '수학 ', '과학 ', '사회 '], dtype='object')

[90 80 70 60 50]
simhyunseok@simhyunseokui-MacBookPro hw_1 %
```

그림 5 Pandas Series 인덱스 활용 실행 결과

결과 설명

scores.values와 subjects.values를 각각 데이터와 인덱스로 전달하여 과목명과 점수를 연결하였다.

scoreSeries["과학"]의 출력은 60이다. index는 과목명 목록, values는 [90, 80, 70, 60, 50]을 반환하였다.

과목명을 이용하니 숫자 위치를 기억하지 않아도 원하는 점수를 찾을 수 있었다.

실습 6 Pandas DataFrame 생성과 조회

컴퓨터 부품 정보를 딕셔너리로 구성한 뒤 DataFrame으로 변환하였다. 전체 표, 특정 열, 앞부분의 행을 차례로 조회하였다.

Python 코드

```
import pandas as pd

data = {
    "MacBook Pro 16": ["CPU", "GPU", "RAM", "SSD"],
    "사양": ["18-core CPU", "20-core GPU", "48GB Unified Memory", "1TB SSD"],
    "비교제품군": [
        "Intel Core Ultra 9",
        "NVIDIA GeForce RTX 5070",
        "48GB LPDDR5X",
        "1TB NVMe SSD",
    ],
}

matrix = pd.DataFrame(data)
print(matrix)
print()
print(matrix["MacBook Pro 16"])
print()
print(matrix.head(2))
```

수행 결과

```
simhyunseok@simhyunseokui-MacBookPro hw_1 % python3 06_pandas_dataframe.py
MacBook Pro 16          사양          비교 제품군
0          CPU          18-core CPU          Intel Core Ultra 9
1          GPU          20-core GPU          NVIDIA GeForce RTX 5070
2          RAM          48GB Unified Memory          48GB LPDDR5X
3          SSD          1TB SSD          1TB NVMe SSD

0          CPU
1          GPU
2          RAM
3          SSD
Name: MacBook Pro 16, dtype: object

MacBook Pro 16          사양          비교 제품군
0          CPU          18-core CPU          Intel Core Ultra 9
1          GPU          20-core GPU          NVIDIA GeForce RTX 5070
simhyunseok@simhyunseokui-MacBookPro hw_1 %
```

그림 6 Pandas DataFrame 생성과 조회 실행 결과

결과 설명

딕셔너리의 키는 열 이름이 되고, 각 리스트의 값은 행 방향으로 배치되었다. matrix["MacBook Pro 16"]을 선택하면 CPU, GPU, RAM, SSD가 들어 있는 Series가 반환된다. head(2)는 CPU와 GPU에 해당하는 처음 두 행을 보여 준다. 이 표는 DataFrame 구조를 연습하기 위한 예시이며, 제품 간 성능이 같다는 의미는 아니다.

2 Quiz 3개 데이터의 누적 막대그래프

강의자로 52쪽의 “3개의 데이터를 누적해서 표현하려면?”이라는 문제를 TIOBE 프로그래밍 언어 순위 데이터로 바꾸어 풀었다. 2022년부터 2025년까지 각 1월의 상위 3개 언어를 한 막대에 표시하였다.

아래부터 3위, 2위, 1위를 쌓았다. 2위 막대의 bottom은 y3이고, 1위 막대의 bottom은 y3와 y2의 원소별 합이다. 리스트끼리 + 연산을 하면 연결되므로, zip()과 리스트 컴프리헨션으로 같은 연도의 값을 더하였다.

Python 코드

```
import matplotlib.pyplot as plt

languages1 = ["Python", "Python", "Python", "Python"]
languages2 = ["C", "C", "C", "C++"]
languages3 = ["Java", "C++", "C++", "Java"]

# TIOBE Rating (%)
y1 = [13.58, 16.36, 13.97, 23.28] # 1st
y2 = [12.44, 16.26, 11.44, 10.29] # 2nd
y3 = [10.66, 12.91, 9.96, 10.15] # 3rd

x = range(len(y1))

bottom1 = [a + b for a, b in zip(y3, y2)]

bar3 = plt.bar(x, y3, width=0.7, color="yellow", label="3rd")
bar2 = plt.bar(x, y2, width=0.7, color="orange", bottom=y3, label="2nd")
bar1 = plt.bar(x, y1, width=0.7, color="red", bottom=bottom1, label="1st")

plt.title("Top 3 Programming Languages by TIOBE Index, January 2022-2025")
plt.xlabel("Year")
plt.ylabel("Popularity (%)")

xLabel = ["2022", "2023", "2024", "2025"]
plt.xticks(x, xLabel, fontsize=10)

plt.legend([bar1, bar2, bar3], ["1st", "2nd", "3rd"])

for i in range(len(y1)):
    # 3위
    plt.text(i, y3[i] / 2, f"{languages3[i]} ({y3[i]}%)", ha="center", va="center")

    # 2위
    plt.text(
        i, y3[i] + y2[i] / 2, f"{languages2[i]} ({y2[i]}%)", ha="center", va="center"
    )

    # 1위
    plt.text(
        i,
        bottom1[i] + y1[i] / 2,
        f"{languages1[i]} ({y1[i]}%)",
        ha="center",
        va="center",
    )

plt.show()
```

Quiz 수행 결과와 풀이 설명

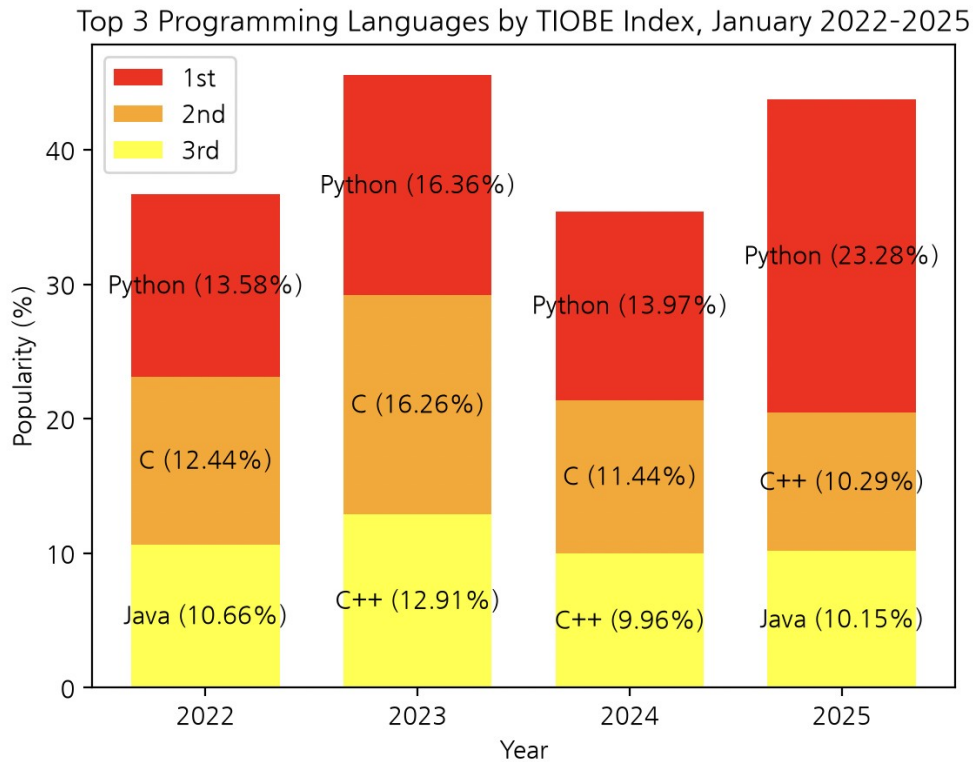


그림 7 TIOBE 상위 3개 언어의 누적 막대그래프

연도	3위 시작 높이	2위 시작 높이	1위 시작 높이	막대 전체 높이
2022	0	10.66	23.10	36.68
2023	0	12.91	29.17	45.53
2024	0	9.96	21.40	35.37
2025	0	10.15	20.44	43.72

예를 들어 2025년에는 Java 10.15 위에 C++ 10.29를 쌓고, 그 합인 20.44부터 Python 23.28을 그린다. 따라서 전체 높이는 43.72가 된다. 그래프에서도 세 구간이 겹치지 않고 순서대로 쌓인 것을 확인할 수 있다. 수치는 모두 TIOBE Rating의 백분율이다.

색의 구분 기준은 언어 자체가 아니라 해당 연도의 순위이다. 2022년의 3위는 Java이고 2023년과 2024년의 3위는 C++이므로, 각 구간 안에 언어명과 값을 함께 넣었다. 흑백으로 보아도 범례의 순위와 구간 안의 글자로 구분할 수 있다.

TIOBE는 언어의 인기를 나타내는 지표이므로 이 값을 실제 개발자의 사용 비율이나 언어의 성능으로 해석하지 않았다. 또한 상위 3개만 표시했기 때문에 전체 높이가 100이 될 필요는 없다.

3 추가 실습 수리력과 소득의 산점도

국가별 평균 수리력 점수와 시간당 소득 사이에 어떤 관계가 있는지 살펴보았다. 두 수치의 관계를 점으로 비교할 수 있는 산점도를 선택했으며, Matplotlib 공식 scatter 예제의 `ax.scatter()` 사용 방법을 참고하였다.

사용 데이터와 처리 방법

OECD PIAAC 2주기 자료를 정리한 `piaac_employed_24.csv`를 사용하였다. CSV에는 24개국의 수리력 점수, 시간당 소득 중앙값, 단위, 조사 시기와 원본 셀 위치가 들어 있다. 조사 시기는 2022~2023년이며, 소득은 구매력평가(PPP)를 반영한 2022년 미국 달러 기준이다.

수리력은 원표 A.4.1 (N)의 취업자 전체 평균으로, 정규교육 중이 아닌 25~65세 성인이 대상이다. 소득은 A.4.7 (N)의 취업자 및 자영업자 시간당 세전 소득 중앙값으로, 보너스를 포함한다. 두 표에서 값이 있는 24개국을 비교했으며, 24개국은 개인 응답자 수가 아니다.

계산에는 CSV의 원래 소수점 값을 사용하였다. `pd.read_csv()`로 읽은 `Numeracy_Score`를 x축, `Wage_Metric`을 y축에 넣고, 딕셔너리를 이용해 국가명을 한글로 표시하였다.

수행 결과

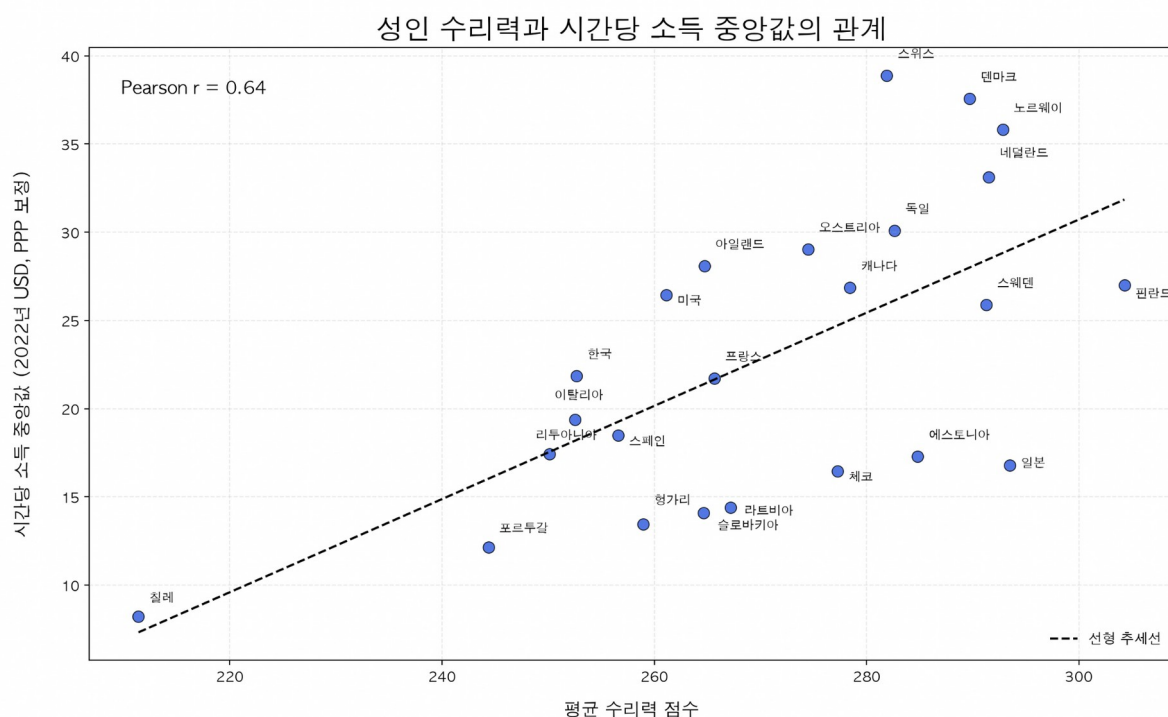


그림 8 24개국 평균 수리력과 시간당 소득 중앙값의 관계

산점도 Python 코드 데이터와 국가명

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# 한글 폰트 설정 (macOS)
plt.rcParams["font.family"] = "AppleGothic"
plt.rcParams["axes.unicode_minus"] = False

# 데이터 불러오기
data = pd.read_csv("piaac_employed_24.csv")

x = data["Numeracy_Score"]
y = data["Wage_Metric"]

# 국가명 한글 변환
country_ko = {
    "Austria": "오스트리아",
    "Canada": "캐나다",
    "Chile": "칠레",
    "Czechia": "체코",
    "Denmark": "덴마크",
    "Estonia": "에스토니아",
    "Finland": "핀란드",
    "France": "프랑스",
    "Germany": "독일",
    "Hungary": "헝가리",
    "Ireland": "아일랜드",
    "Italy": "이탈리아",
    "Japan": "일본",
    "Korea": "한국",
    "Latvia": "라트비아",
    "Lithuania": "리투아니아",
    "Netherlands": "네덜란드",
    "Norway": "노르웨이",
    "Portugal": "포르투갈",
    "Slovak Republic": "슬로바키아",
    "Spain": "스페인",
    "Sweden": "스웨덴",
    "Switzerland": "스위스",
    "United States": "미국",
}
```

코드 설명

CSV 파일은 코드와 같은 폴더에 두고 hw_1 폴더에서 실행하였다. AppleGothic은 macOS에서 한글을 표시하기 위한 폰트 설정이다.

산점도 Python 코드 라벨과 점 표시

```
# 겹침 완화를 위한 국가별 라벨 위치 조정값
# (x방향, y방향) in points
offsets = {
    "Chile": (8, 8),
    "Portugal": (8, 8),
    "Hungary": (8, 12),
    "Lithuania": (-10, 8),
    "Latvia": (10, -8),
    "Slovak Republic": (10, -15),
    "Spain": (8, -10),
    "Italy": (-15, 12),
    "France": (8, 10),
    "Korea": (8, 10),
    "Czechia": (8, -10),
    "Estonia": (8, 10),
    "Japan": (8, -4),
    "United States": (8, -10),
    "Ireland": (8, 10),
    "Austria": (8, 10),
    "Germany": (8, 10),
    "Canada": (8, 10),
    "Sweden": (8, 10),
    "Finland": (8, -12),
    "Netherlands": (8, 12),
    "Norway": (8, 10),
    "Denmark": (8, 10),
    "Switzerland": (8, 10),
}

# 그래프 생성
fig, ax = plt.subplots(figsize=(13, 8))

# 산점도 점 표시
ax.scatter(x, y, s=70, color="royalblue", edgecolor="black", linewidth=0.6, alpha=0.9)

# 국가명 라벨 표시
for _, row in data.iterrows():
    country = str(row["Country"])
    korean_name = country_ko.get(country, country)

    dx, dy = offsets.get(country, (6, 6))

    ax.annotate(
        korean_name,
        (row["Numeracy_Score"], row["Wage_Metric"]),
        xytext=(dx, dy),
        textcoords="offset points",
        fontsize=9,
        ha="left",
        va="bottom",
    )
```

코드 설명

offsets는 데이터 좌표를 바꾸는 값이 아니라 국가명 글자의 위치를 점 단위로 조정하는 값이다. annotate()를 이용해 각 점에 국가명을 붙였다.

산점도 Python 코드 추세선과 상관계수

```
# 선형 추세선 계산
slope, intercept = np.polyfit(x, y, 1)
x_line = np.linspace(x.min(), x.max(), 100)
y_line = slope * x_line + intercept

# 추세선 표시
ax.plot(
    x_line, y_line, color="black", linestyle="--", linewidth=1.6, label="선형 추세선"
)

# Pearson 상관계수
r = np.corrcoef(x, y)[0, 1]
ax.text(
    0.03,
    0.95,
    f"Pearson r = {r:.2f}",
    transform=ax.transAxes,
    fontsize=13,
    verticalalignment="top",
)

# 제목 및 축 이름
ax.set_title("성인 수리력과 시간당 소득 중앙값의 관계", fontsize=20)

ax.set_xlabel("평균 수리력 점수", fontsize=13, labelpad=10)

ax.set_ylabel("시간당 소득 중앙값 (2022년 USD, PPP 보정)", fontsize=13, labelpad=20)

# 격자
ax.grid(linestyle="--", alpha=0.25)

# 추세선 범례
ax.legend(loc="lower right", fontsize=10, frameon=False)

# 여백
fig.subplots_adjust(left=0.12, right=0.97, top=0.90, bottom=0.12)

plt.show()
```

코드 설명

`np.polyfit(x, y, 1)`로 직선의 기울기와 절편을 구하고, `np.corrcoef(x, y)[0, 1]`로 Pearson 상관계수를 계산하였다. 점선은 24개국에 같은 가중치를 둔 선형 추세선이다.

산점도 결과 해석과 실습 정리

산점도 결과 해석

계산된 Pearson 상관계수는 약 0.6411이며, 그래프에는 소수 둘째 자리까지 표시해 $r = 0.64$ 로 나타냈다. 선택한 24개국에서는 평균 수리력이 높을수록 시간당 소득 중앙값도 높은 편이었다. 다만 점들이 추세선 위에 일정하게 놓이지는 않았다.

일본은 평균 수리력 점수가 약 293.51로 높지만, 시간당 소득 중앙값은 약 16.78달러였다. 반면 스위스는 수리력 점수가 약 281.90으로 일본보다 낮아도 소득 중앙값은 약 38.89달러로 높았다. 이처럼 수리력 하나만으로 국가별 소득 차이를 설명하기는 어렵다.

이 그래프는 국가 단위 집계값의 관계를 보여 준다. 따라서 개인이 수리력 점수를 높이면 소득이 반드시 증가한다고 말할 수는 없다. 국가별 산업 구조나 임금 수준 등 다른 요인을 따로 고려하지 않았고, 표본오차를 반영한 추론도 하지 않았으므로 전체적인 분포를 살펴보는 수준으로 해석하였다.

실습 정리

NumPy에서는 배열을 만드는 것뿐 아니라 같은 원소를 다른 모양으로 바꾸거나 일부만 선택하는 과정을 확인하였다. Pandas에서는 Series의 인덱스와 DataFrame의 열 이름을 이용하면 원하는 값을 찾기 편하다는 점을 확인할 수 있었다.

누적 막대그래프에서는 bottom에 어떤 값을 전달하는지가 핵심이었다. 산점도에서는 CSV를 읽어 그래프를 그리는 데서 더 나아가 국가명, 추세선, 상관계수를 함께 표시하였다. 그래프를 완성하는 것과 그 결과를 적절하게 해석하는 것은 별개의 과정이라는 점도 확인하였다.

참고자료

1. 김봉재, 「ch04 파이썬 프로그래밍 기초」, 2026학년도 강의자료. NumPy 및 Pandas 실습 43~48쪽, Quiz 52~53쪽, 과제 안내 63쪽.
 2. Matplotlib, scatter(x, y). 산점도의 ax.scatter() 사용 방법 참고.
https://matplotlib.org/stable/plot_types/basic/scatter_plot.html
 3. TIOBE, TIOBE Index. 지표의 정의 및 프로그래밍 언어 인기 지수. 그래프에는 2022~2025년 각 1월의 값을 사용하였다.
<https://www.tiobe.com/tiobe-index/>
 4. OECD, Survey of Adult Skills 2023 관련 통계표. A.4.1 (N)의 B열과 A.4.7 (N)의 AB열 사용.
<https://stat.link/files/b263dc5d-en/smlvj6.xlsx>
 5. 데이터 파일 piaac_employed_24.csv. Country, Numeracy_Score, Wage_Metric 등 7개 열과 24개국 자료. Source_Numeracy와 Source_Wage 열에 원본 표와 셀 위치 기록.
- 웹 자료 확인일 2026년 9월 22일